Assignment 1

Step 0

Before proceeding with the exercise please implement the steps 1 to 4 from the OpenFlow tutorial [3]. This will help you deal with the tutorial pre-requisites, install required software, download and setup the Mininet VM and learn the development and debugging tools, such as wireshark, dpctl, etc. This is very important to continue.

Introduction

In this exercise, you will learn about the open-source OpenFlow controller POX. You will learn how to write network applications, i.e., Hub and Layer 2 MAC Learning etc., on POX and run them on a virtual network based on Mininet. Hub and MAC learning examples will only serve for getting familiar with POX: the final objective of the exercise is to write yourself a load balancer application which uses an OpenFlow switch to balance load stemming from a set of clients towards a set of servers (4 clients-4 servers in particular). More details on creating and submitting the code will be provided later on in the instructions. So, make sure that you follow each step carefully.
(Note: you can skip this section and start directly with the assignment at page 7, if you feel confident and are already familiar with POX and its basic functions)

Getting Familiar with POX

Invoking the hub application

POX is a Python based SDN controller platform geared towards research and education. Cur rently it supports OpenFlow v1.0 [7] and this is the version that will be used for this exercise. For more details on POX, see POX wiki [4]. To start, navigate to the ˜/pox directory in your Mininet VM, pull the latest code and checkout the *carp* branch:

```
$ cd ~/pox
$ git pull
$ git checkout carp
$ cd ~
```

We are going to use POX as the exercise's OpenFlow controller. We will not use the reference controller anymore, which is the default controller that Mininet uses during its simulation. So, make sure that it's not running in the background:

```
$ ps -A | grep controller
```


If so, you should kill it; either press Ctrl-C in the window running the controller program, or from another SSH window:

```
$ sudo killall controller
```

In case a running controller is not killed because you have invoked its CLI, just press Ctrl-D in the CLI window. You should also run:

```
sudo mn -c
```

and restart Mininet to make sure that everything is clean and using the faster kernel switch. From your Mininet VM run:

$ sudo mn --topo single,8 --mac --switch ovsk --controller remote

This will start the Mininet network emulator that we will use for experimentation. Essentially, this command tells Mininet to start up an 8-host network with a single switch (running Open vSwitch [2]), set the MAC address of each host equal to its IP for readability, and point to a remote controller, which by default runs in localhost. Here is what Mininet just did:

- Created 8 virtual hosts, each with a separate IP address.
- Created a single OpenFlow software switch with 8 ports, running in the kernel space of Mininet.
- Connected each virtual host to the switch with a virtual ethernet cable. •

  Set the MAC address of each host equal to its IP address.
- Configured the OpenFlow switch to connect to a remote controller (running in localhost in our case).

The POX controller comes pre-installed with the provided VM image. Now, from another SSH window connected to the same Mininet vm, run the basic hub example, after going to the ~/pox directory:

$ cd ~/pox
$ ./pox.py log.level --DEBUG forwarding.hub

This tells POX to enable verbose logging (for debugging) and to start the hub component. The switch of your topology might take a little bit of time to contact POX. This is because, when an OpenFlow switch loses its connection to a controller, it will generally increase the period between which it attempts to contact the controller again, up to a maximum of 15 seconds. Since the OpenFlow switch has not connected yet, this delay may be anything between 0 and 15 seconds. If this is too long to wait, the switch can be configured to wait no more than N seconds using the max-backoff parameter. Alternately, you exit Mininet to remove the switch(es), start the controller, and then start Mininet to immediately connect. Wait until the application indicates that the OpenFlow switch has connected to POX. When the switch connects, POX will print something like this:

POX 0.0.0 / Copyright 2011 James McCauley
INFO:forwarding.hub:Hub running.
DEBUG:core:POX 0.0.0 going up...
DEBUG:core:Running on CPython (2.7.3/Sep 26 2012 21:51:14)
INFO:core:POX 0.0.0 is up.
This program comes with ABSOLUTELY NO WARRANTY. This program is free software, 2

and you are welcome to redistribute it under certain conditions. Type
'help(pox.license)' for details.
DEBUG:openflow.of_01:Listening for connections on 0.0.0.0:6633 Ready.
POX> INFO:openflow.of_01:[Con 1/1] Connected to 00-00-00-00-00-01
INFO:forwarding.hub:Hubifying 00-00-00-00-00-01

Verifying hub behavior with tcpdump

Now verify that hosts can ping each other, and that all hosts see the exact same traffic - the behavior of a hub. To do this, we'll create xterms for each host and view the traffic in each. In the Mininet console, start up three xterms:

mininet> xterm h1 h2 h3

Arrange each xterm so that they're all on the screen at once. This may require reducing the height to fit a cramped laptop screen. In the xterms for h2 and h3, run tcpdump, a utility to print the packets seen by a host:

tcpdump -XX -n -i h2-eth0

and respectively:

tcpdump -XX -n -i h3-eth0

In the xterm for h1, send a ping:

ping -c1 10.0.0.2

The ping packets are now going through the switch, which then floods them out of all its ports except the sending one. You should see identical ARP and ICMP packets corresponding to the ping in both xterms running tcpdump. This is how a hub works; it sends all packets to every port on the network. All the hub flow rules have been installed by the controller on startup. Now, see what happens when a non-existent host doesn't reply. From h1 xterm:

ping -c1 10.0.0.10

You should see three unanswered ARP requests in the xterms running tcpdump.You can use this technique to debug your code later (essentially, three unanswered ARP requests is a signal that you might be accidentally dropping packets. You can close the xterms now.

Taking a closer look at POX-based network programming

First, let's look at the hub code:

```
from pox.core import core
import pox.openflow.libopenflow_01 as of
from pox.lib.util import dpidToStr

log = core.getLogger()

def _handle_ConnectionUp (event):
  msg = of.ofp_flow_mod()
  msg.actions.append(of.ofp_action_output(port = of.OFPP_FLOOD))
  event.connection.send(msg)
  log.info(''Hubifying %s'', dpidToStr(event.dpid))
```



```
def launch ():
  core.openflow.addListenerByName("ConnectionUp", _handle_ConnectionUp)

  log.info("Hub running.")
```

The following API primitives are useful for building such network applications on POX. All these primitives can be accessed after importing the openflow library of POX:

import pox.openflow.libopenflow_01 as of

Some examples are the following:

- connection.send(...) function sends an OpenFlow message to a switch. When a connection to a switch starts, a ConnectionUp event is fired. The above code invokes a handle ConnectionUp () function that implements (for example) the hub logic.
- ofp action output class: This is an action for use with ofp packet out and ofp flow mod. It specifies a switch port that you wish to send the packet out of. It can also take various ”special” port numbers. An example of this, as shown in the hub example, would be OFPP FLOOD which sends the packet out all ports except the one the packet originally arrived on.

  Example: Create an output action that would send packets to all ports:

  ```
  out_action = of.ofp_action_output(port = of.OFPP_FLOOD)
  ```

- ofp match class (not used in the code above but might be useful in the exercise): Objects of this class describe packet header fields and an input port to match on. All fields are optional – items that are not specified are“wildcarded” and will match on anything. Some notable fields of ofp match objects are:

  – dl src - The data link layer (MAC) source address
  – dl dst - The data link layer (MAC) destination address
  – in port - The packet input switch port

  Example: Create a match that matches packets arriving on port 3:

  ```
  match = of.ofp_match()
  match.in_port = 3
  ```

- ofp packet out OpenFlow message (not used in the code above but might be useful in the exercise): The ofp packet out message instructs a switch to send a packet. The packet might be one constructed at the controller, or it might be one that the switch received, buffered, and forwarded to the controller (and is now referenced by a buffer id).
  Notable fields are:

  – buffer id - The buffer id of a buffer you wish to send. Do not set if you are sending a constructed packet.
  – data - Raw bytes you wish the switch to send. Do not set if you are sending a buffered packet.
  – actions - A list of actions to apply (for this tutorial, this is just a single ofp action output action).

  – in port- The port number this packet initially arrived on if you are sending by buffer id, otherwise OFPP NONE.

- ofp flow mod OpenFlow message: This instructs a switch to install a flow table entry. Flow table entries match some fields of incoming packets, and execute some list of actions on matching packets. The actions are the same as for ofp packet out, mentioned

above (and, again, for the tutorial all you need is the simple ofp action output action). The match is described by an ofp match object.
Notable fields are:

- idle timeout - Number of idle seconds before the flow entry is removed. Defaults to no idle timeout.
- hard timeout - Number of seconds before the flow entry is removed. Defaults to no timeout.
- actions - A list of actions to perform on matching packets (e.g., ofp action output).
- priority - When using non-exact (wildcarded) matches, this specifies the priority for overlapping matches. Higher values are higher priority. Not important for exact or non-overlapping entries.
- buffer id - The buffer id of a buffer to apply the actions to immediately. Leave unspecified for none.
- in port - If using a buffer id, this is the associated input port.
- match - An ofp match object. By default, this matches everything, so you should probably set some of its fields!

Example: Create a flow mod that sends packets from port 3 out of port 4:

```
fm = of.ofp_flow_mod()
fm.match.in_port = 3
fm.actions.append(of.ofp_action_output(port = 4))
```

Invoking the learning switch application and verifying behavior with tcpdump

This time, lets verify that hosts can ping each other when the controller is behaving like a Layer 2 learning switch. Kill the POX controller by pressing Ctrl-C in the window running the controller program. In case the running controller is not killed because you have invoked its CLI, just press Ctrl-D in the CLI window. Now run the l2 learning example:

```
$ ./pox.py log.level --DEBUG forwarding.l2_learning
```

Like before, we'll create xterms for each host and view the traffic in each. In the Mininet console, start up three xterms:

```
mininet> xterm h1 h2 h3
```

Arrange each xterm so that they're all on the screen at once. This may require reducing the height to fit a cramped laptop screen. In the xterms for h2 and h3, run tcpdump, a utility to print the packets seen by a host:

```
# tcpdump -XX -n -i h2-eth0
```

and respectively:

```
# tcpdump -XX -n -i h3-eth0
```


In the xterm for h1, send a ping:

```
# ping -c1 10.0.0.2
```

Here, the switch examines each packet and learns the source-port mapping. Thereafter, the

source MAC address will be associated with the port. If the destination of the packet is already associated with some port, the packet will be sent to the given port, else it will be flooded on all ports of the switch. You can close the xterms now. The code for l2 learning application is provided under ~/pox/pox/forwarding. The learning switch module will be the main example that will help you solve the current exercise.